

Kelvin Ngeno

AIT 512

Uber Pathfinder

1. Program Requirements

This project models an Uber-like dispatch system in a small urban area, where intersections are represented as graph nodes and roads as undirected edges. The objective is to find the shortest route a driver can take from a pickup point (origin) to a drop-off point (destination) using Breadth First Search (BFS). The application uses a graph of 6 vertices simulating neighborhoods. Each edge denotes a two-way road, ensuring bidirectional travel. This scenario is common in urban mobility planning, where the shortest route must be computed to optimize fuel, time, and customer satisfaction. In real time ridesharing, mapping such graphs and computing shortest paths quickly is essential. The use of BFS ensures the shortest path is always found (assuming uniform cost between intersections), which is suitable for city grids or where distance/cost is not weighted.

2. Relevance to Topic Area

This project is directly relevant to graph theory and search algorithms covered in this semester. It involves the use of an undirected graph to represent a real-world taxi dispatch network and employs BFS, a classic graph traversal algorithm. The solution simulates a real problem that uses core computing concepts: graph modeling, adjacency lists, and queue-based traversal. This reinforces understanding of search mechanisms and graph data structures, which are critical for many applications in AI, transportation networks, networking, and even social media connections. Moreover, implementing this in Java using modular object-oriented design (OOP) aligns perfectly with course goals and my 502 class.

3. Implementation Explanation

The Java program is structured into three classes under the package. The Graph.java class initializes an adjacency list representation (lines 9–13) and defines methods like addEdge(int v, int w) and getNeighbors(int v) to connect vertices. The BFSPathFinder.java class contains the core logic for BFS traversal (lines 13–37), initializing visited[], parent[], and a queue. It then reconstructs the shortest path (lines 39–44) and prints the result. The Main.java file executes the program by building a graph (lines 12–18), initializing BFSPathFinder, and calling printShortestPath(0, 5) (line 20). The entire flow

mimics a GPS routing engine in rideshare platforms. The algorithm prints the path: [0, 1, 3, 5], clearly showing how the system finds the optimal route from start to end.

4. Graph Representation (30 points)

The graph is represented using an adjacency list, which is optimal for sparse graphs and saves memory compared to adjacency matrices. Each vertex maps to a list of neighbors, stored using a `Map<Integer, List<Integer>>` structure (line 10 in `Graph.java`). This design allows fast lookup and easy iteration over neighbors, essential for BFS. Since roads in the Uber system are bidirectional, the use of an undirected graph is appropriate. The `addEdge()` method (line 14) ensures both directions are stored, i.e., `graph[v].add(w)` and `graph[w].add(v)`. This representation is simple, efficient, and scales well with additional intersections.

5. Search Algorithm

The program uses BreadthFirst Search (BFS) to find the shortest path between two vertices. BFS is ideal for unweighted graphs where the shortest path equals the fewest number of edges. The algorithm uses a queue to explore the graph level-by-level (lines 18–28 in `BFSPathFinder.java`). For every visited vertex, its neighbors are added to the queue. The parent array stores the previous vertex, which helps reconstruct the path. This ensures the path found is indeed the shortest. DFS would not guarantee the shortest path in an unweighted graph, so BFS is the correct and efficient choice here.

6. Execution

The graph is displayed using adjacency list format.

BFS is executed from node 0 to node 5.

Output: Shortest path: [0, 1, 3, 5]

This confirms the algorithm correctly traced the shortest route from pick-up to destination via intersections.

Graph.class — src

OPEN EDITORS

UberCityNavigator.javaGraph.classGraph.java 1BFSPathFinder.java

OPEN EDITORS

UberCityNavig...Graph.classGraph.java... 1BFSPathFinder...

SRC

m5b/d1ca

BFSPathFinder...BFSPathFinder...Graph.classGraph.java 1UberCityNavig...UberCityNavig...

4import java.util.Iterator;

5import java.util.LinkedList;

6import java.util.List;

7

8public class Graph {

9private final int vertices;

10private final List<Integer>[] adjList;

11

12public Graph(int var1) {

13this.vertices = var1;

14this.adjList = new LinkedList<Integer>[var1];

15

16for(int var2 = 0; var2 < var1; ++var2) {

17this.adjList[var2] = new LinkedList<Integer>();

18}

19

20}

21

22public void addEdge(int var1, int var2) {

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

GITLENS

COMMENTS

zsh

kelvinkibet@Kelvins-MacBook-Pro-2 ~/Desktop/AIT 512/src

main

javac m5b/d1ca/*.java

Note: m5b/d1ca/Graph.java uses unchecked or unsafe operations.

Note: Recompile with -Xlint:unchecked for details.

kelvinkibet@Kelvins-MacBook-Pro-2 ~/Desktop/AIT 512/src

main

java m5b.d1ca.UberCityNavigator

Program: Uber City Navigator

Author: Kelvin Ngeno

Date of Execution: 2025-11-12 22:24:25

Graph connections:

Vertex 0 connects to: 1 2

Vertex 1 connects to: 0 3

Vertex 2 connects to: 0 4

Vertex 3 connects to: 1 5

Vertex 4 connects to: 2

Vertex 5 connects to: 3

Finding shortest path from 0 to 5:

Shortest path: [0, 1, 3, 5]

kelvinkibet@Kelvins-MacBook-Pro-2 ~/Desktop/AIT 512/src

main

OUTLINE

main*

Launchpad

0 1

Indexing completed.

Java: Ready

Ln 1, Col 1

Spaces: 3

Java

Prettier